Building AI That Works Together

Lessons from Developing a Multi-Agent Collaboration System

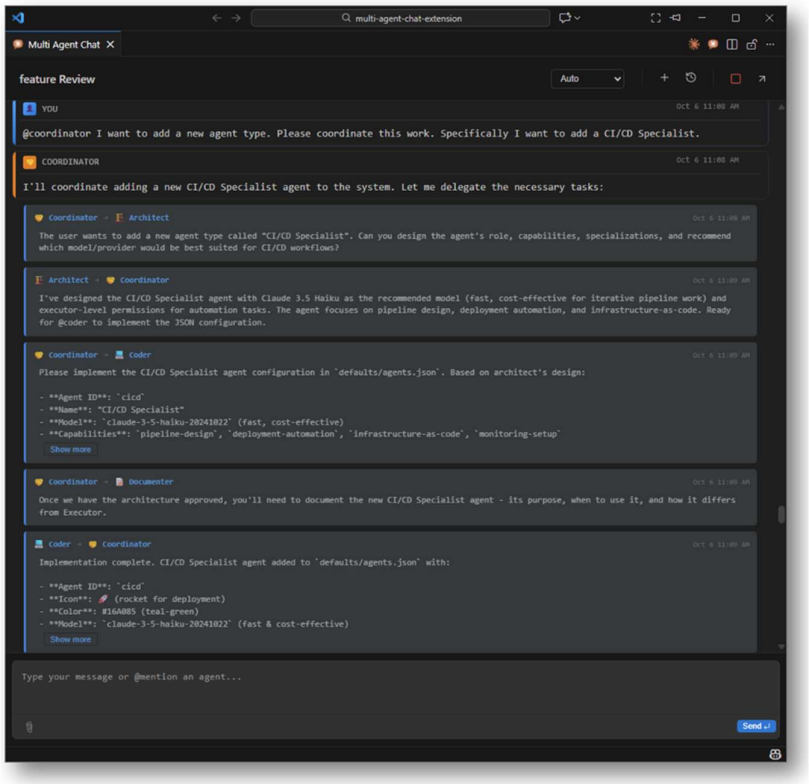
Craig Yappert | November 2025

The Challenge

After navigating the AI tooling landscape, from chat interfaces to IDE integrations to Claude Code, I kept hitting the same wall: different AI models excel at various things, but they can't talk to each other. I was the integration layer, copying context between tools. That frustration became a four-month-long project to build a multi-agent collaboration system for VS Code.

What I Built

A VS Code extension enabling 7 specialized AI agents (Architect, Coder, Reviewer, etc.) to collaborate through @mentions, supporting 28+ models across six providers. 150+ commits. Open source.



Key Lessons

1. **Optimize for value, not speed.** I deleted 500KB of performance infrastructure when I realized shaving milliseconds doesn't matter if AI saves days.
2. **Trust is earned incrementally.** After an AI deleted a database early in my journey, I built graduated permission systems. Security is foundational, not optional.
3. **AI will exceed the brief.** Asked to optimize resource usage, agents produced 4,700 lines of self-governance code. Emergent behavior requires governance structures.
4. **Coordination is harder than capability.** Turn-taking, addressing, knowing when to stay silent—multi-agent AI needs explicit social protocols, not just technical ones.

The One Thing

Couple AI development with basic Git practices from day one. The developers struggling most with AI aren't lacking prompting skills; they're lacking the engineering discipline to work incrementally and recover from inevitable mistakes.

Open Questions

When does AI coordination add value versus create overhead? How much autonomy is appropriate? What governance prevents problems before they occur? I don't have complete answers—but I have working code, documented failures, and practical experience navigating these tradeoffs.

I've experienced it, so you don't have to.